

1. a) What is embedded system?

Explain the different characteristics of embedded system?

Name

Class

Ser

Roll No

Subje.

Additional Sheet No

Invigilator

Sol:

An embedded system is a ~~specific system~~ special purpose system in which the computer is completely encapsulated by the device it controls.

The different characteristics of an embedded system are as follows:

i) Single functioned:

→ An embedded system usually executes a program specific program repeatedly. So, it must be single functioned.

ii) Tightly Constrained:

→ All computing systems have constraint on design metrics but those on embedded system can be ~~little~~ tightly especially tight.

iii) Real Time and Reactive:

→ An embedded system continuously reacts in many changes in system environments and compute the certain result in real time without delay.

iv) Microprocessors based:

→ An embedded system must be microprocessor or microcontroller based.

v> Memory:

→ An embedded system must have a memory, as its software usually embeds in ROM.

vi> Connected:

→ An embedded system must have connected peripherals to connected input and output device.

vii> Hardware and Software Systems:

→ In embedded system, software is used for more features and flexibility.

→ In embedded system, hardware is used for performance and security.

Name Python, NAST
 Class Ser Roll No
 Subj.:
 Additional Sheet No
 invigilator

2.6> What is ASIP? Explain the necessity of ASIP comparing with SPP and GPP.

- Sol:
- ASIP stands for Application - Specific Instruction Set Processors.
 - An ASIP is a programmeable architecture that is designed in a specific way to perform certain tasks more efficiently.
 - ASIPs provides an intermediate solution between GPPs and SPPs because they are designed for an application.
 - ASIPs can be programmed by writing software resulting in short time to market and good flexibility.
 - ASIPs tend to come in three major varieties: microcontrollers, DSPs and less-general ASIPs.

Features	SPP	ASIP	GPP
Full Form	Single Purpose Processor	Application specific Instruction-set Processor.	General Purpose Processor.
Flexibility	Very low	Moderate	Very high
Efficiency	High	High	Moderate
Power Consumption	Very low	High	Low
Complexity	High	Moderate	High
cost	High	Moderate	High

3.6) Why is interrupt important? Compare and contrast interrupt and DMA.

~~3.6~~ An interrupt is a signal emitted by a device attached to a computer or from a program within the computer.

External devices are comparatively slower than CPU.

If there is no interrupt, CPU would waste a lot of time waiting for external devices to match its speed with that of CPU. This decreases the efficiency of CPU. Hence, interrupt is required to eliminate these limitations. Thus, interrupt is important.

* Advantages of Interrupt:

- i) It increases the efficiency of the CPU.
- ii) It decreases the waiting time of the CPU.
- iii) It stops the wastage of instruction cycle.
- iv) It simplifies I/O operations.

S.N	Interrupt	DMA
i>	Interrupt alerts the CPU to handle specific event immediately.	DMA transfer data between memory and devices without knowing the CPU.
ii>	Interrupt requires CPU intervention to handle the interrupt.	DMA minimizes CPU involvement during data transfer.
iii>	Interrupt is efficient for handling events.	DMA is efficient large data transfer.
iv>	Interrupt involves context switching and ISR execution.	DMA requires initial setup by the CPU.
v>	Interrupt can temporarily slow down the CPU due to context switching.	DMA improves overall system performance by freeing up the CPU.
vi>	Interrupt is used for handling real-time events, quick responses to peripheral actions.	DMA is used for managing large data transfers without overloading the CPU.
vii>	Interrupt is used for event handling.	DMA is used for efficient data transfer.

4.a) What is semaphore? How semaphore can be used for global resource sharing?

Ans:

→ A semaphore is a kernel object that one or more thread of execution can acquire or release for the purpose of synchronization or mutual exclusion.

→ Semaphore can be used for global resource sharing by following ways:-

i) Initialization:

→ A semaphore is initialized with a count representing the number of available resources.

→ For global resource sharing, this count is set to the number of instances of the resources that can be accessed concurrently.

ii) Acquire (P operation):

→ When a process or thread wants to access the shared resource, it performs an acquire operation of the semaphore.

→ If the semaphore count is greater than zero, the count is decremented and access is granted.

→ If the semaphore count is zero, the ~~count is de~~ process or thread is blocked until the semaphore count becomes positive.

iii) Release (V operation):

- When a process or thread finishes using the resource, it performs a release operation on the semaphore.
- This increments the semaphore count and if, there are any processes or threads waiting, allows one of them to acquire the resource.

4.b) Explain memory fragmentation. How problem related to memory fragmentation can be solved in embedded system?

89
21

→ Memory fragmentation refers to the condition where free memory is divided into small, ^{non-}contiguous blocks over time, making it difficult to allocate large contiguous blocks of memory.

→ There are two types of memory fragmentation:

i) External fragmentation

ii) Internal fragmentation.

→ The problem related to memory fragmentation can be solved in embedded system by following ways:

i) Paging:

→ A paging is a technique which is used for contiguous memory allocation where the program is divided into fixed size pages.

→ For paging, operating system is accountable.

→ Paging could result ⁱⁿ internal fragmentation.

ii) Segmentation:

→ A segmentation is a technique which is used for non-contiguous memory allocation where the program is divided into variable size sections.

→ For segmentation, compiler is accountable.

→ Segmentation could result in external fragmentation.

5.6) Define debugger, downloader, ~~cross~~-compiler and cross-assembler.

sp

i) Debugger:

- A debugger is a computer program that is used to test and debug other programs.
- The debugger can be used to identify if the program is running correctly and cause of the failure if it fails.
- The debugger runs on the development processor and supports stepwise program execution.
- Debugger is also known as Instruction Set Simulator (ISS) and Virtual Machine (VM).
- The debugger may be a source level debugger or a low level debugger.

ii) Downloader:

- Downloader is used to download, execute and debug embedded software over communication link.
- Downloader is also known as remote debugger.
- Downloader downloads the binary image to the embedded system.
- Downloader supports high level of interaction between host and target.
- Usually, downloader have front end and backend interfaces.

iii) Cross-Compiler:

- A cross-compiler translates high level programming language into equivalent assembly language program of a target processor.
- A cross-compiler takes an ~~embe~~ expanded source program written high in high level language as an input.
- A cross-compiler is capable of creating executable code for a platform other than one on which the compiler is running.
- A cross-compiler is necessary to compile for multiple platforms from one machine.
- For example : In order to compile Linux/ARM, we first need to obtain its libraries to compile against.

iv) Cross-assembler:

- A cross-assembler converts source program written in the assembly language into the machine language of a target processor.
- Cross-assemblers are used to develop program which will run on game console and other small electronic system which are not able to run development environment on their own.
- Cross-assembler can be used to give speed development on low powered system.
- C 64 is the best example of cross-assembler.

6. a) What are different styles of modeling technique used in VHDL? Describe them.

sg The different styles of modeling technique used in VHDL are described below:

i) Dataflow style of modeling:

→ Dataflow modeling can be described based on the Boolean expression.

→ Dataflow modeling shows how the data flows from input to output.

→ Dataflow modeling works on concurrent execution.

ii) Behavioral style of modeling:

→ Behavioral modeling is used to execute statements sequentially.

→ Behavioral modeling shows how the system performs according to the current statement.

→ Behavior modeling may contain process statements, sequential statements, signal assignment statements & wait statements.

iii) Structural style of modeling:

→ Structure modeling is used to specify the functionality and structure of the system.

→ In structural modeling, an entity is described as a set of interconnected components.

→ Structural modeling, contain signal declaration, component instances and portmap in component instances.

7. Write short notes on;

a) Arbitration:

→ Arbitration is the process when more than one module controlling the bus.

→ for example: CPU and DMA controller.

→ In arbitration, only one module may control bus system at a time.

→ There are three centralized bus arbitration methods:

i) Daisy chaining method:

→ In daisy chaining method, all bus masters use the same line for making bus requests.

ii) Polling method:

→ In polling method, the controller is used to generate the address for the master, the number of lines

→ required depends on the number of masters connected in the system.

iii) Independent Request method:

→ In independent request method, each master has a separate pair of bus request and bus grant lines and each pair has a priority assigned to it.

b) Memory write and storage performance:

89

* Memory write:

i) High end:

→ Example: Registers, RAMs etc.

ii) Middle Range:

→ Example: FLASH, EEPROM etc.

iii) Lower Range:

→ Example: UV EPROM, ~~OTP~~ OTPROM etc.

iv) Low END:

→ Example: mask-programmed ROM

* Storage Performance:

i) High End:

→ Example: mask-programmed ROM, OTPROM,

ii) Middle Range:

→ Example: NVRAM, FLASH etc

iii) Lower Ranger:

→ Example: SRAM

iv) Low End:

→ Example: DRAM